Lecture#3
SOCKET Programming

Dr.Negm Shawky

EMAIL: negmshawky@gmail.com

Contents

- Introduction
- Networking Basics
- Understanding Ports and Sockets
- Java Sockets
  - Implementing a Server
  - Implementing a Client
- Sample Examples
- Conclusions

Basics

SOCKET PROGRAMMING



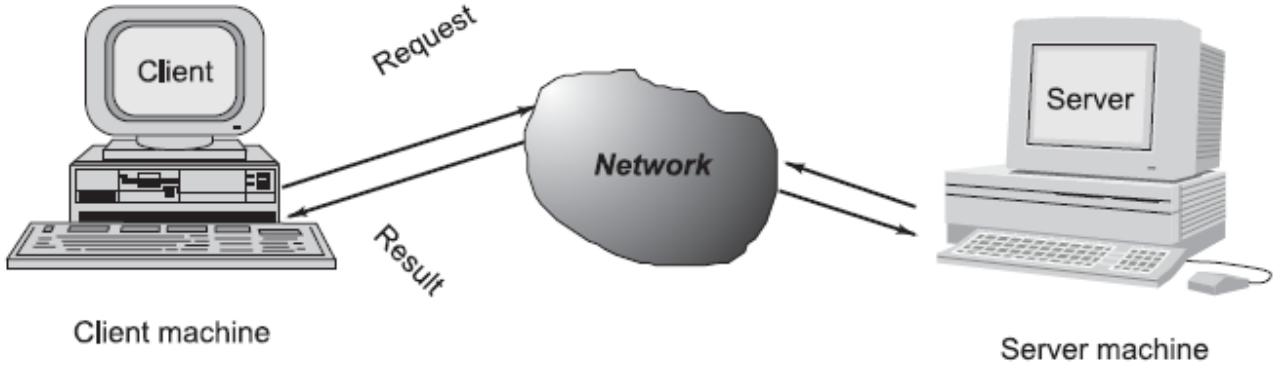
How you can concentrate in the lecture

overview

- In this lecture we present key concepts of intercommunication between programs running on different computers in the network.
- This is based on creating network applications using sockets.
- This require java.net package containing various classes for creating sockets and message communication using two different protocols.

Client/Server Communication

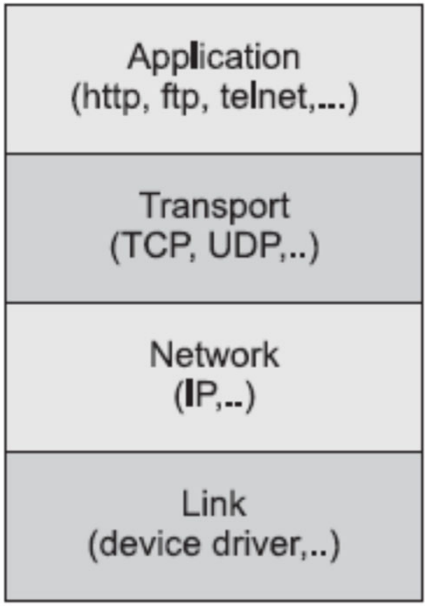
- We studied before in lecture of networking theory, the network-based systems consist of a server , client , and a medium for communication
- A computer running a program that makes a request for services is called client machine.
- A computer running a program that responses to the requested services from one or more clients is called server machine.
- The medium for communication can be wired or wireless network.



Client/Server Communication

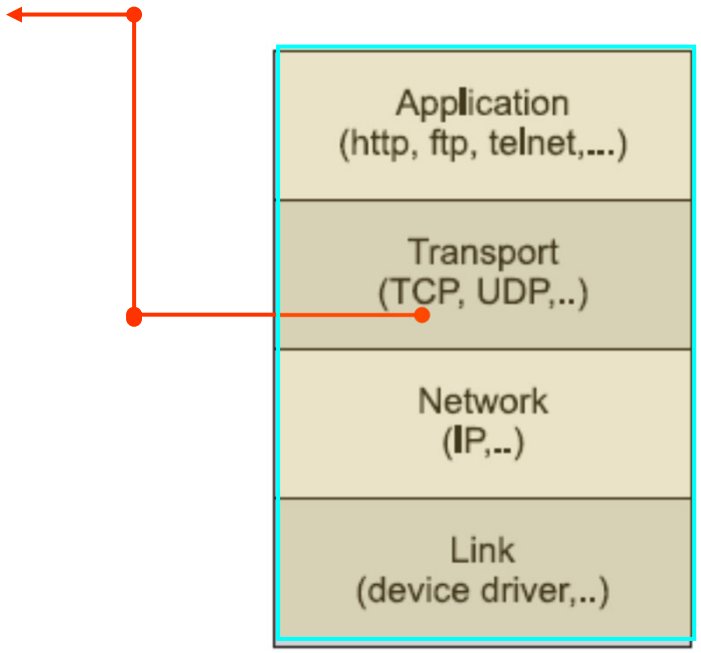
- programs running on client machines make requests to a program running on a server machine (often called as server program).
- The communication between the running programs involve networking services provided by the transport layer.

- The transport layer comprises two types of protocols, TCP (Transmission Control Protocol) and UDP (User Datagram Protocol).
- sockets are used for programming interfaces based on these protocols.



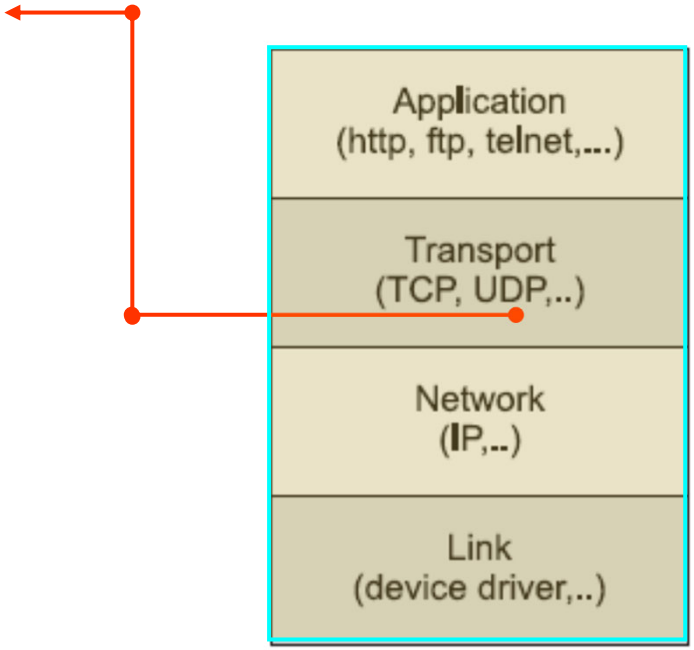
Networking Basics

- TCP (Transmission Control Protocol) is a connection-oriented communication protocol that provides a reliable flow of data between two computers
- Example applications:
  - HTTP
  - FTP
  - Telnet



Networking Basics

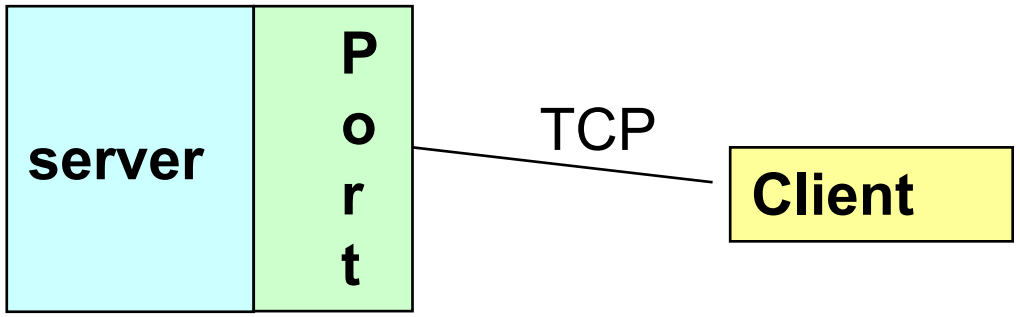
- UDP (User Datagram Protocol) is a connectionless communication protocol that sends packets of data, called *datagram*, from one computer to another with no guarantee about arrival

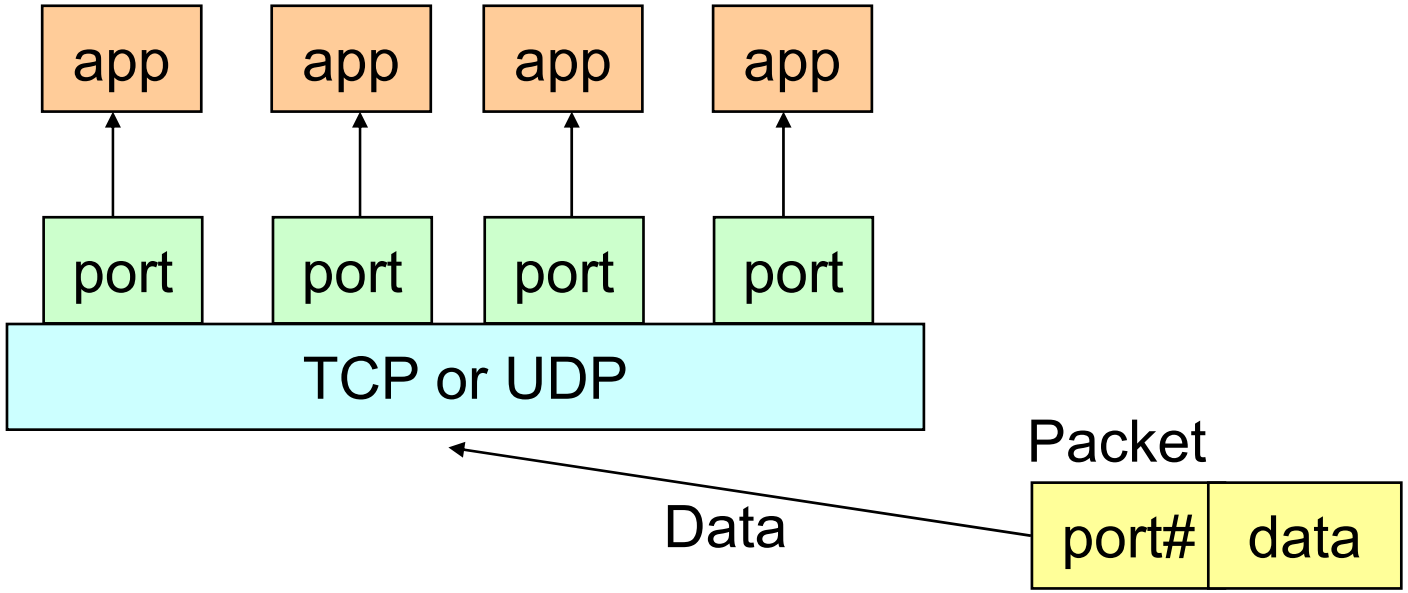


- This Similar to sending multiple emails/letters to friends.
- Example applications:
  - Media streaming
  - Clock server
  - Ping (Ping is the name of a standard software utility used to test network connections

Understanding Ports

- The TCP and UDP protocols use *ports* to map incoming data to a particular *process* running on a computer.





Understanding Ports

- Port is represented by a positive (16-bit) integer value
- Some ports have been reserved to support common/well known services:

  ftp 21/tcp

  telnet 23/tcp

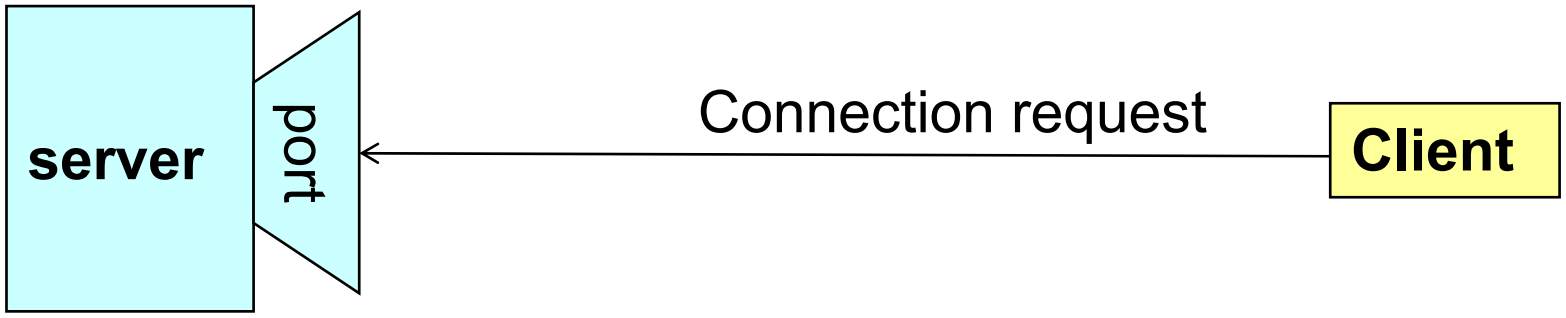
  smtp 25/tcp

  login 513/tcp

  http 80/tcp,udp

Socket Communication

- A server (program) runs on a specific computer and has a socket that is bound to a specific port.
- The server waits and listens to the socket for a client to make a connection request.
- If everything goes well, the server accepts the connection.



Sockets and Java Socket Classes

- A socket is an endpoint of a two-way communication link between two programs running on the network.
- A socket is bound to a port number so that the TCP layer can identify the specific type of application that data intended to be sent.
- Java.net package provides two key classes used in creation of Socket for server and client programs are:

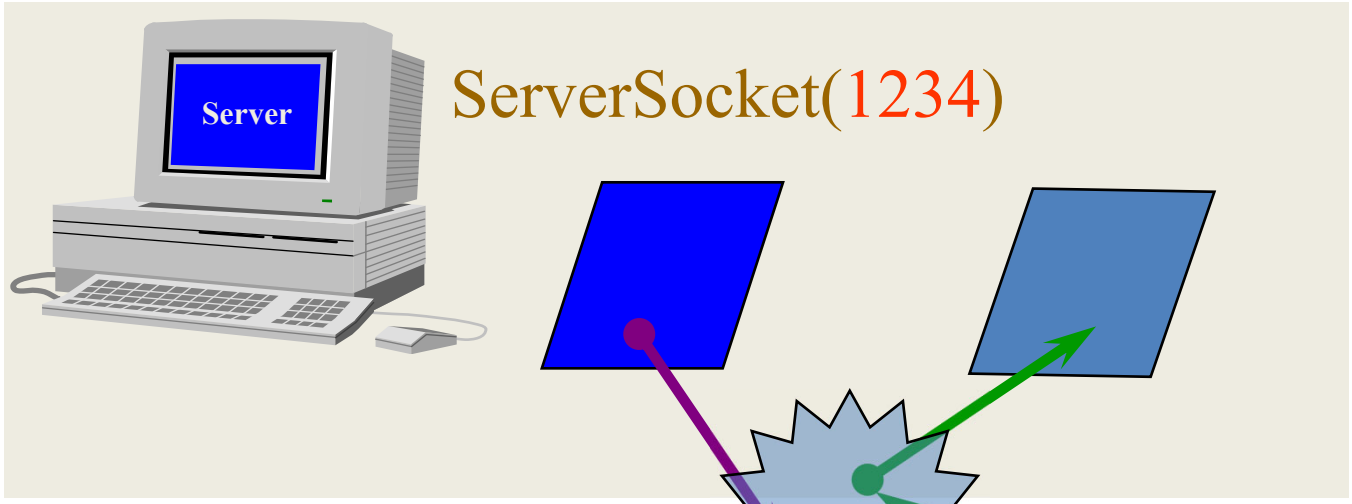
  ServerSocket – for implementing a server

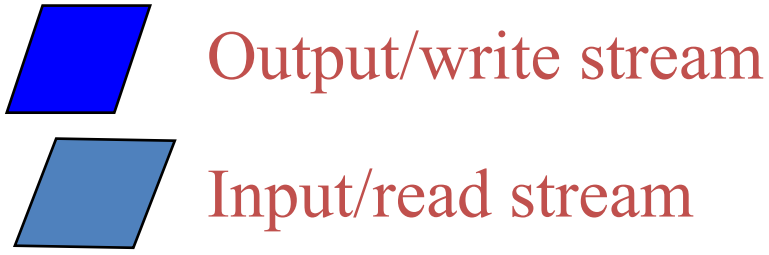
  Socket – for implementing a client

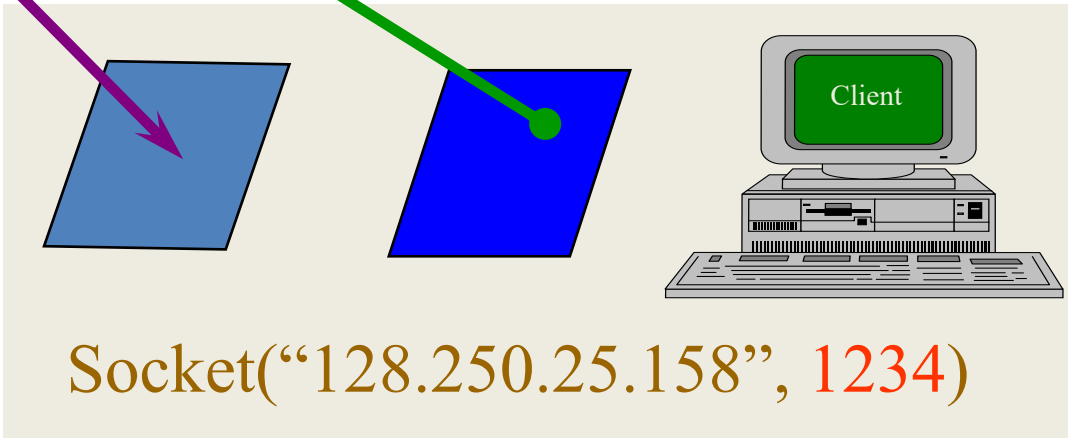
TCP/IP SOCKET PROGRAMMING

- A server program creates a specific type of socket that is used to listen for client requests (server socket),
- In the case of a connection request is established, the server program uses the socket through which it will exchange data with the client using input and output streams.
- The socket abstraction is very similar to the file concept (developers have to open a socket, perform I/O, and close it).
- The following figure illustrates key steps involved in creating socket based on server and client programs.

Java Sockets







It can be host_name like "mandroo.cs.mu.oz.au"

Implementing a Server

1. Open the Server Socket:

```
ServerSocket server;
DataOutputStream os;
DataInputStream is;
server = new ServerSocket( PORT );
```

2. Wait for the Client Request:

```
Socket s_server = server.accept();
```

3. Create I/O streams for communicating to the client

```
is = new DataInputStream(
s_server.getInputStream() );
os = new DataOutputStream(
s_server.getOutputStream() );
```

Implementing a Server

4. Perform communication with client

```
Receive from client:
String line = is.readLine();
Send to client:
os.writeBytes("Hello\n");
```

5. Close sockets:

```
s_server.close();
```

Implementing a Client

1. Create a Socket Object:

```
client = new Socket( server, port_id );
```

2. Create I/O streams for communicating with the server.

```
is = new DataInputStream(
client.getInputStream() );
```

```
os = new DataOutputStream(
client.getOutputStream() );
```

3. Perform I/O or communication with the server:

```
Receive data from the server:
    String line = is.readLine();
Send data to the server:
    os.writeBytes("Hello\n");
```

4. Close the socket when done:

```
client.close();
```

A simple server (simplified code)

- An example program illustrating creation of a server socket, waiting for client request, and then responding to a client that requested for connection :

```
// SimpleServer.java: a simple server program
import java.net.*;
import java.io.*;
public class SimpleServer {
  public static void main(String args[]) throws
IOException {
    // Register service on port 1234
    ServerSocket s = new ServerSocket(1234);
    Socket s_c = s.accept(); // Wait and accept
a connection
```

A simple server (simplified code)

```
// Get a communication stream associated with
the socket
    OutputStream s_cout = s_c.getOutputStream();
    DataOutputStream dos = new
     DataOutputStream (s_cout);
    // Send a string!
    dos.writeUTF("Hi there");
    // Close the connection, but not the
     server socket
    dos.close();
    s_cout.close();
    s_c.close();
  }
}
```

A simple client (simplified code)

- An example program illustrating establishment of connection to a server and then reading a message sent by the server and displaying it on the console is given below:

```
// SimpleClient.java: a simple client program
import java.net.*;
import java.io.*;
public class SimpleClient {
  public static void main(String args[])
throws IOException {
    // Open your connection to a server, at
port 1234
    Socket sc = new
Socket("mundroo.cs.mu.oz.au",1234);
```

A simple client (simplified code)

```
    // Get an input file handle from the
socket and read the input
    InputStream scIn = sc.getInputStream();
    DataInputStream dis = new
     DataInputStream(scIn);
    String st = new String (dis.readUTF());
    System.out.println(st);
    // When done, just close the connection
and exit
    dis.close();
    scIn.close();
    sc.close();
  }
}
```

Run

- To illustrate execution of server and client programs,

let us assume that a machine called **mundroo.cs.mu.oz.au** on which we want to run a server program as indicated below:

Run Server on mundroo.cs.mu.oz.au

[raj@mundroo] java SimpleServer &

Let us say we want to run our client program on a machine called gridbus.csse.unimelb.edu.au as follows:

Run Client on any machine (including gridbus):

[raj@gridbus] java SimpleClient

Hi there

Socket Exceptions

```
try {
        Socket client = new Socket(host, port);
        handleConnection(client);
}
catch(UnknownHostException uhe) {
        System.out.println("Unknown host: " + host);
        uhe.printStackTrace();
}
catch(IOException ioe) {
        System.out.println("IOException: " + ioe);
        ioe.printStackTrace();
}
```

IOException - if an I/O error occurs when opening the socket

Server in Loop: Always up

```
// SimpleServerLoop.java: a simple server program that runs forever in a single thread
import java.net.*;
import java.io.*;
public class SimpleServerLoop {
  public static void main(String args[]) throws IOException {
    // Register service on port 1234
    ServerSocket s = new ServerSocket(1234);
```

Server in Loop: Always up

```
while(true)
   {
        Socket sc=s.accept(); // Wait and accept a connection
        // Get a communication stream associated with the
         socket
        OutputStream scout = sc.getOutputStream();
        DataOutputStream dos = new DataOutputStream
         (scout);
        // Send a string!
        dos.writeUTF("Hi there");
        // Close the connection, but not the server socket
        dos.close();
        scout.close();
        sc.close();
    }
  }
}
```

LECTURE 3 - PART A | PROGRAM — ARCHITECTURAL FLOW

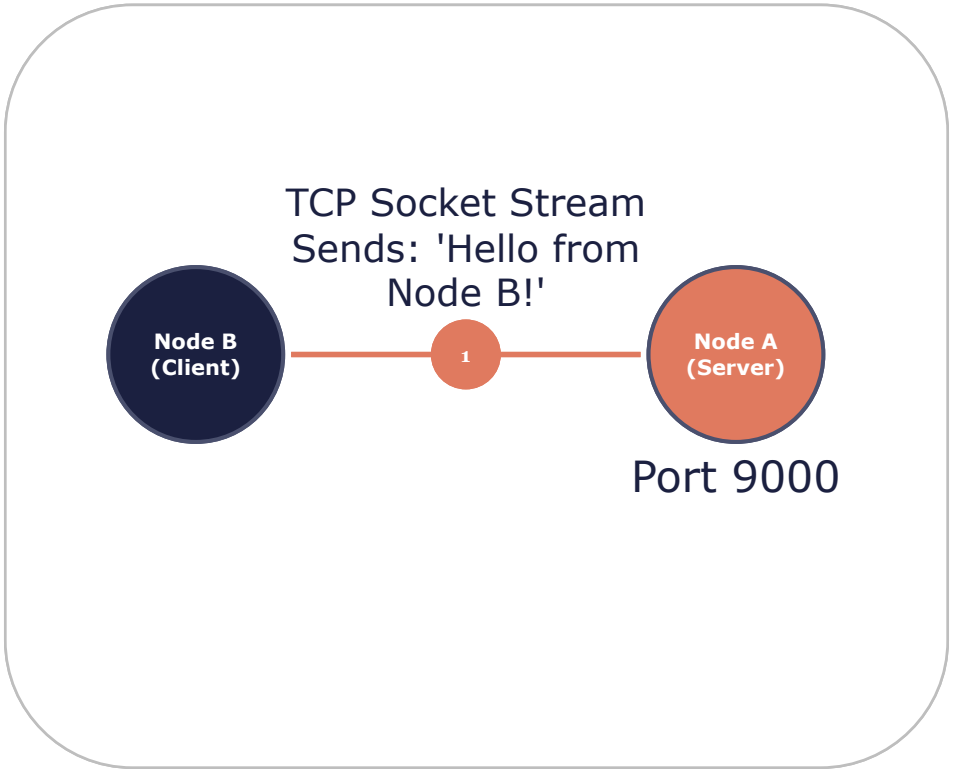
TCP Sockets enable direct node-to-node message passing

SIMPLE DESCRIPTION

Here is a simple program that demonstrates Interprocess Communication (IPC) using Message Passing over TCP Sockets. It consists of two independent programs:

Node A (Server): Waits for incoming connections and receives a message.

Node B (Client): Connects to the server and sends a message.

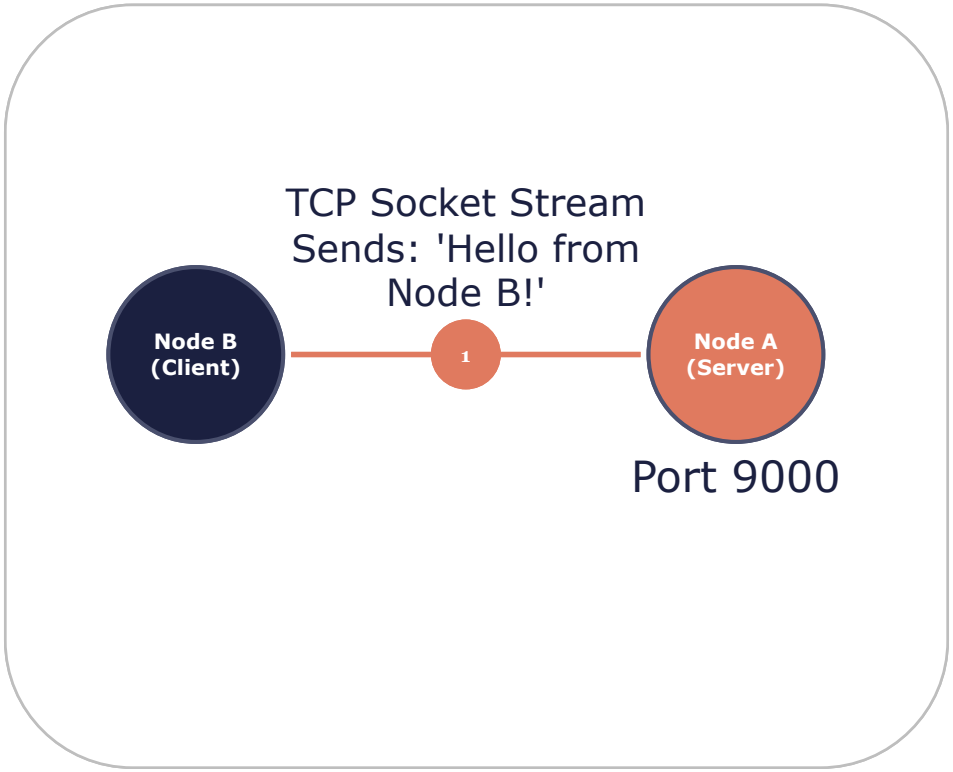


LECTURE 3 - PART A | PROGRAM — ARCHITECTURAL FLOW

TCP Sockets enable direct node-to-node message passing

FUNCTION OF THE PROGRAM

This program demonstrates Interprocess Communication (IPC) in a distributed system. Two separate processes running independently communicate through TCP sockets. The client (Node B) sends a text message to the server (Node A), and the server receives and displays the message, illustrating the concept of message passing between distributed nodes over a network.



```
1. Server Program (Node A)
import  .io.*;
import  .net.*;
public class NodeAServer {
public static void main(String[] args) {
  try {
// Create a server socket on port 9000
   ServerSocket server = new
ServerSocket(9000);
   System.out.println("Node A is waiting for a
connection...");
 // Wait for a client to connect
  Socket connection = server.accept();
 // Read message from the client
  BufferedReader in = new BufferedReader(
  new
InputStreamReader(connection.getInputStrea
m()));
  String message = in.readLine();
  System.out.println("Received: " + message);
  // Close connection
  connection.close();
  server.close();
  }
  catch (Exception e)
{
   e.printStackTrace();}}}
```

How the Program Works

Step 1: Start Node A (Server)
The server opens port 9000 and waits for a connection.
Node A is waiting for a connection...

Step 2: Start Node B (Client)
The client connects to Node A and sends:
Hello from Node B!

Step 3: Server Receives the Message
Node A receives the message and displays it.
Received: Hello from Node B!

2. Client Program (Node B)

```
import  .io.*;
import  .net.*;
public class NodeBClient {
public static void main(String[] args) {
try {
// Connect to Node A
    Socket socket = new
Socket("localhost", 9000);
  // Send message to the server
    PrintWriter out = new
PrintWriter(socket.getOutputStream(),
true);
    out.println("Hello from Node B!");
    System.out.println("Message sent to
Node A.");
    // Close connection
    socket.close();
    }
catch (Exception e)
{
   e.printStackTrace();
  }
   }
 }
```

Output on Node A (Server)

```
1 -Node A is waiting for a connection...
2 -Received: Hello from Node B!
```

Output on Node B (Client)

```
Message sent to Node A.
```

LECTURE 3 - PART A | PROGRAM — ARCHITECTURAL FLOW

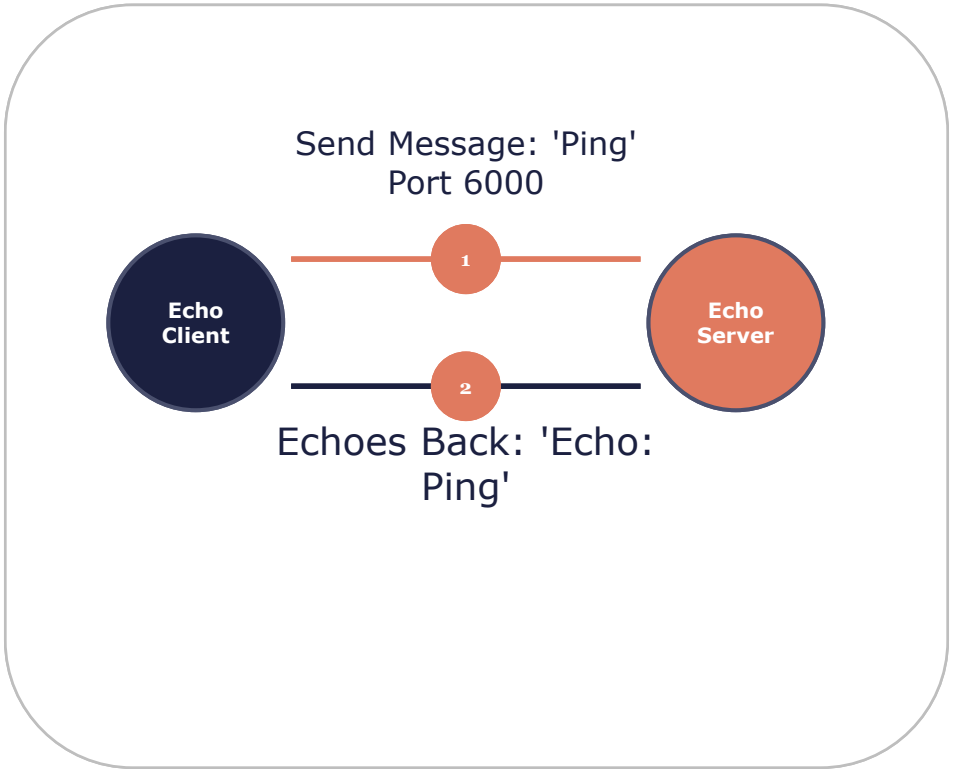
Echo Server receives messages and returns them back

SIMPLE DESCRIPTION

Here is a simple program that demonstrates a Client-Server Architecture and Message Passing using TCP Sockets. The server acts as an Echo Server, which receives a message from the client and sends it back with the prefix "Echo: ".
Server Program (EchoServer.)

FUNCTION OF THE PROGRAM

This simple example illustrates how a client and server communicate through TCP sockets, with the server echoing back the client's message.



How the Program Works

- The server starts and listens on port 6000.
- The client connects to the server.
- The client sends the message: Ping
- The server receives the message and creates a response:
  Echo: Ping
- The server sends the response back to the client.
- The client receives and displays the response.

```
server Program (Echoserver. )
import  .io.*;
import  .net.*;
public class EchoServer {
public static void main(String[] args) {
try {
    // Create server socket on port 6000
      ServerSocket server = new ServerSocket(6000);
      System.out.println("Server is waiting for a client...");
    // Accept client connection
      Socket client = server.accept();
    // Input stream to receive data
      BufferedReader in = new BufferedReader(
        new InputStreamReader(client.getInputStream()));
    // Output stream to send data
      PrintWriter out = new PrintWriter(client.getOutputStream(), true);
   // Read message from client
      String message = in.readLine();
    // Send echo response
      out.println("Echo: " + message);
    // Close connections
      client.close();
      server.close();
   }
 catch (Exception e) {
      e.printStackTrace();
 }}}
```

Client Program (EchoClient.)

```
import  .io.*;
import  .net.*;
public class EchoClient {
public static void main(String[] args) {
try {
// Connect to the server
      Socket socket = new Socket("localhost", 6000);

// Output stream to send data
      PrintWriter out = new
PrintWriter(socket.getOutputStream(), true);
  out.println("Ping");
// Input stream to receive response
     BufferedReader in = new BufferedReader(
  new InputStreamReader(socket.getInputStream()));
String message = in.readLine();
  System.out.println(message);
```

Output on the Server Screen

Server is waiting for a client...

Output on the Client Screen

Echo: Ping

Final Result

Echo: Ping

UDP socket Programming

- Datagram packets are used to implement a connectionless packet delivery service supported by the UDP protocol.
- Each message is transferred from source machine to destination based on information contained within that packet.
- That means, each packet needs to have destination address.
- Packet delivery is not guaranteed.

UDP socket Programming

- The format of datagram packet is

| Msg | length | Host | serverPort |

- Java supports datagram communication through the following classes:

  - DatagramPacket ():
  - DatagramSocket

UDP socket Programming

- The class DatagramPacket contains several constructors that can be used for creating packet object.

  One of them is:

```
DatagramPacket(byte[] buf, int length, InetAddress address, int port);
```

- // This constructor is used for creating a datagram packet for sending packets of length to the specified port number on the specified host.
- The message to be transmitted is indicated in the first argument.

- The key methods of DatagramPacket class are:

  byte[] getData()

  //Returns the data buffer.

  int getLength()

  //Returns the length of the data to be sent or the length of the data received.

UDP socket Programming

void setData(byte[] buf)
//Sets the data buffer for this packet.
void setLength(int length)
//Sets the length for this packet.

- The class DatagramSocket supports various methods that can be used for transmitting or receiving a datagram over the network.
- The two key methods are:

void send(DatagramPacket p)
//Sends a datagram packet from this socket.
void receive(DatagramPacket p)
//Receives a datagram packet from this socket.

UDP socket Programming

To send and receive a message to a specified hostname on port using UDP.

1. **Import the necessary classes**: DatagramPacket, DatagramSocket, and InetAddress from the java.net package.
2. **Define the message, length, address, and port**: Convert the message to bytes, get the InetAddress of the server, and specify the port number.
3. **Create a DatagramPacket**: This packet will contain the message, its length, the server address, and the port number.
4. **Create a DatagramSocket**: This socket will be used to send the packet.
5. **Send the packet**: Use the send method of the DatagramSocket to send the packet.
6. **Close the socket**: Always close the socket after use to free up resources.

UDP socket Programming

- A corresponding **client program** for creating a datagram and then sending it to the above server and then accepting a response is listed below.

UDP Client: Sends a Message and Gets reply

```
import java.net.*;
import java.io.*;
 public class UDPClient
  {
   public static void main(String args[]){
      // args give message contents and server hostname
      DatagramSocket dSocket = null;
       try {
            dSocket = new DatagramSocket();
            byte [] m = args[0].getBytes();
            InetAddress Host = InetAddress.getByName(args[1]);
            int serverPort = 6789;
            DatagramPacket request = new DatagramPacket(m,
             args[0].length(), Host, serverPort);
            dSocket.send(request);
            byte[] buffer = new byte[1000];
```

UDP Client: Sends a Message and Gets reply

```
DatagramPacket reply = new DatagramPacket(buffer, buffer.length);
            dSocket.receive(reply);
            System.out.println("Reply: " + new String(reply.getData()));
        }
        catch (SocketException e){
          System.out.println("Socket: " + e.getMessage());}
        catch (IOException e){
          System.out.println("IO: " + e.getMessage());}
}
}
```

UDP socket Programming

- A simple **UDP server program** that waits for client's requests and then accepts the message (datagram) and sends back the same message is given below.

- Of course, an extended server program can manipulate client's request messages and send a new message as a response.

UDP Sever: repeatedly received a request and sends it back to the client

```
import java.net.*;
import java.io.*;
public class UDPServer{
      public static void main(String args[]){
      DatagramSocket dSocket = null;
         try{
            dSocket = new DatagramSocket(6789);
            byte[] buffer = new byte[1000];
            while(true){
               DatagramPacket request = new DatagramPacket(buffer,
               buffer.length);
              dSocket.receive(request);
              DatagramPacket reply = new DatagramPacket (
                        request.getData(), request.getLength(),
                        request.getAddress(), request.getPort());
```

UDP Sever: repeatedly received a request and sends it back to the client

```
            dSocket.send(reply);
              }
            }catch (SocketException e){
            System.out.println("Socket: " + e.getMessage());}
            catch (IOException e) {
            System.out.println("IO: " +e.getMessage());}
}
}
```

END

